

VULNERABILITY ASSESSMENT & PENETRATION TESTING REPORT

Vishal Gold — Android Mobile Application

Project	Vishal Gold — B2B/B2C Jewelry App
Prepared For	Vishal Gold Management & Technical Stakeholders
Prepared By	Senior Android VAPT Expert — Engineering Security Team
Version	v2.0 (Post-Remediation)
Date	February 25, 2026

Document Control

Attribute	Detail
Document Title	Vulnerability Assessment and Penetration Testing Report
Target Application	Vishal Gold — B2B/B2C Jewelry Mobile Application (Android)
Document Version	v2.0 — Post-Remediation Final
Classification	CONFIDENTIAL — Restricted Distribution
Assessment Date	February 2026
Report Date	February 25, 2026
Lead Assessor	Senior Android VAPT Expert, Engineering Security Team
Reviewed By	Engineering Team Lead
Distribution	Vishal Gold CTO, Project Manager, Engineering Lead

Version History

Version	Date	Author	Description
v1.0	February 20, 2026	Security Team	Initial assessment report — pre-remediation findings
v1.1	February 22, 2026	Security Team	Updated findings following Iteration 1 remediation
v2.0	February 25, 2026	Security Team	Final post-remediation report with retest results

This document is classified CONFIDENTIAL. It contains detailed technical vulnerability information that could enable attacks if disclosed inappropriately. Distribution is restricted to named recipients only. Recipients are responsible for ensuring that this document is stored securely and not forwarded to unauthorised parties.

Table of Contents

1. Executive Summary
2. Scope and Objectives
3. Rules of Engagement and Assumptions
4. Methodology and Standards
5. Target Environment Details
6. Risk Rating Methodology
7. Detailed Findings
8. Consolidated Risk Summary
9. Remediation Roadmap and Prioritisation
10. Retest Validation Results
11. Conclusion
12. Appendix A — Tools Used
13. Appendix B — References and Standards
14. Appendix C — Evidence and Artefact Notes

1. Executive Summary

The Engineering Security Team conducted a comprehensive Vulnerability Assessment and Penetration Testing (VAPT) exercise against the Vishal Gold Android mobile application, a B2B and B2C commerce platform serving gold jewellery wholesalers, retailers, and administrators. The assessment was performed in two phases: an initial black-box and grey-box assessment against the pre-remediation build, followed by a targeted retest against the post-remediation build to validate all applied fixes.

The initial assessment identified seventeen findings across five severity tiers: three Critical, five High, four Medium, three Low, and two Informational. These vulnerabilities represent real and exploitable risks to the business, including potential for unauthorised administrative access, exfiltration of user personally identifiable information (PII), privilege escalation by regular users, and unauthenticated database manipulation.

Following Iteration 1 of the remediation programme, seven findings were resolved and verified closed. All three Critical findings were remediated. The retest confirmed that all remediated items are no longer exploitable. At the time of this final report, eight findings remain open — two High, two Medium, two Low, and two Informational — each with a documented remediation recommendation and a prioritised target sprint.

The most significant residual risk is the absence of deployed Firestore database security rules (VAPT-006), which currently permits any authenticated user to read or write arbitrary Firestore documents. This finding is classified High and is a prerequisite action before the application is opened to production users. The Engineering team has prepared and provided the required rules, and deployment is a client-side action estimated to take under thirty minutes.

Overall Risk Posture (at Report Date)

Metric	Pre-Remediation	Post-Remediation	Change
Total Findings	17	8	-9 resolved
Critical	3	0	All resolved ✓
High	5	2	3 resolved
Medium	4	2	2 resolved
Low	3	2	1 resolved
Informational	2	2	No action required
Overall Risk Rating	CRITICAL	HIGH	Improved — not yet production-ready

RECOMMENDATION: The application must not be opened to production users until VAPT-006 (Firestore security rules) is deployed. Once this is complete, the residual risk rating will reduce to MEDIUM and the application will be suitable for controlled production launch.

2. Scope and Objectives

2.1 Assessment Objectives

The primary objective of this assessment was to identify, evaluate, and document all security vulnerabilities present in the Vishal Gold Android application, with particular focus on vulnerabilities relevant to the OWASP Mobile Top 10 and the OWASP Mobile Application Security Verification Standard (MASVS). Secondary objectives included verifying the effectiveness of implemented security controls, providing actionable remediation guidance, and producing a client-ready report suitable for management review.

2.2 In-Scope Systems

Asset	Type	Version / Identifier
Vishal Gold Android Application	Mobile APK	Version 1.0 — Debug & Release builds
Firebase Authentication	Cloud Service	Firebase Auth SDK 22.x
Cloud Firestore	Cloud Database	Firebase SDK — default instance
Firebase Storage	Cloud File Store	Firebase Storage SDK
Firebase Cloud Messaging	Push Service	FCM SDK
Android Device (Test)	Physical Device	Android 13, API 33
Android Emulator (Test)	Virtual Device	Android 10, API 29

2.3 Out-of-Scope Systems

The following systems were explicitly excluded from the assessment scope: Google Firebase platform infrastructure (assessed by Google under their shared responsibility model), iOS platform (not in project scope), the Google Play Store distribution platform, and any third-party payment or ERP systems not integrated with the current build.

3. Rules of Engagement and Assumptions

3.1 Rules of Engagement

The following rules governed the conduct of this assessment. Testing was performed exclusively against designated test environments and test accounts. No production Firestore data belonging to real users or customers was read, modified, or deleted. All proof-of-concept demonstrations were performed in an isolated manner to confirm exploitability without causing any operational disruption.

Denial-of-service testing, mass automated scanning, and social engineering attacks were not performed as part of this engagement. The assessment team operated within the boundaries of the application and its backend services. All findings were immediately disclosed to the project team upon discovery using the responsible disclosure protocol.

3.2 Testing Approach

Phase	Approach	Description
Phase 1 — Static Analysis	White-box	Full access to Flutter source code, AndroidManifest.xml, and Firestore configuration
Phase 2 — Dynamic Analysis	Grey-box	Testing with authenticated test accounts (Retailer, Wholesaler, Admin roles)
Phase 3 — Network Analysis	Black-box	Traffic interception via proxy (Charles Proxy / mitmproxy)
Phase 4 — Binary Analysis	White-box	APK decompilation and analysis of compiled artefacts
Phase 5 — Retest	Targeted	Verification of remediated findings against post-fix build

3.3 Assumptions

The following assumptions were made for the duration of this assessment: the Firebase project was in its default configuration as shipped with the application; the google-services.json file provided to the assessment team was the same file used in the development build; and test accounts were representative of the access levels available to real users in the production environment.

4. Methodology and Standards

This assessment was conducted in accordance with recognised industry standards and frameworks for mobile application security testing. The methodology combined

automated static analysis tools with manual expert review, dynamic runtime testing, and a structured retest phase. All findings were rated and reported in accordance with the severity classification system defined in Section 6 of this report.

Standard / Framework	Applicability
OWASP Mobile Top 10 (2024)	Primary classification framework for mobile application vulnerabilities
OWASP MASVS v2.0 (Mobile Application Security Verification Standard)	Verification checklist for security controls at L1 (standard) and L2 (defence-in-depth)
OWASP MSTG (Mobile Security Testing Guide)	Detailed test case library and testing techniques
NIST SP 800-163 (Vetting the Security of Mobile Applications)	Supplementary guidance for government-grade mobile security review
PTES (Penetration Testing Execution Standard)	Overall engagement structure and reporting standard
CVSS v3.1 (Common Vulnerability Scoring System)	Quantitative vulnerability severity scoring methodology

4.1 Testing Phases

Reconnaissance & Threat Modelling

Identification of the application's attack surface, data flows, trust boundaries, and potential threat actors. STRIDE methodology was applied to enumerate threats across spoofing, tampering, repudiation, information disclosure, denial of service, and elevation of privilege categories.

Static Application Security Testing (SAST)

Review of Flutter/Dart source code for hardcoded secrets, insecure API usage, improper data storage, missing input validation, and debug artifacts. AndroidManifest.xml was reviewed for over-permissive declarations and backup settings.

Dynamic Application Security Testing (DAST)

Runtime testing with authenticated and unauthenticated sessions. Network traffic was intercepted and analysed for sensitive data exposure, improper TLS configuration, and API security issues. Authentication and authorisation flows were fully tested.

Binary and Storage Analysis

APK unpacking and analysis of compiled classes, assets, and native libraries. On-device storage was examined for plaintext credentials, session tokens, and cached sensitive data using Android Debug Bridge (ADB).

Firestore Security Analysis

Direct evaluation of Firestore database access control rules, collection structure, and access patterns to identify privilege escalation and unauthorised data access vectors.

5. Target Environment Details

Component	Detail
Application Name	Vishal Gold
Platform	Android (minSdkVersion 21 — Android 5.0+)
Framework	Flutter 3.32 / Dart 3.9
Primary Language	Dart
Backend	Google Firebase (Auth, Firestore, Storage, FCM)
State Management	Provider (ChangeNotifier pattern)
Local Storage (PII)	flutter_secure_storage v9.x — AES-256
Local Storage (Non-PII)	shared_preferences
Networking	Firebase SDK — REST over TLS 1.3
Authentication	Firebase Auth — Phone OTP + Email/Password
User Roles	Retailer, Wholesaler, Administrator
Test Device 1	Physical Android 13 (API 33) device
Test Device 2	Android Emulator — Android 10 (API 29)
Assessment Build	Debug APK (SAST/DAST) + Release APK (binary analysis)

6. Risk Rating Methodology

Each vulnerability identified during this assessment was assigned a severity rating based on two principal factors: the likelihood of successful exploitation and the potential business impact if exploitation were to occur. The combination of these factors produces a qualitative severity rating of Critical, High, Medium, Low, or Informational. Where applicable, CVSS v3.1 base scores are provided as a quantitative reference and are noted within each finding.

Severity	CVSS v3.1 Range	Definition	SLA (Remediation)
Critical	9.0 – 10.0	Immediate, direct threat to business confidentiality, integrity, or availability. Exploitation requires minimal skill and can be performed remotely.	Immediate (≤ 24 hours)
High	7.0 – 8.9	Significant business risk with high impact or high likelihood of exploitation. Requires	72 hours

		authentication or specific conditions.	
Medium	4.0 – 6.9	Moderate risk with limited scope of impact or requires complex exploitation chain.	7 business days
Low	0.1 – 3.9	Minimal risk; exploitation is difficult, impact is limited, or mitigated by compensating controls.	30 days
Informational	N/A	Best-practice observation or configuration improvement with no direct security risk.	Next scheduled maintenance cycle

7. Detailed Findings

This section provides a detailed account of each vulnerability identified during the assessment. Findings are presented in order of descending severity. Each entry includes the OWASP Mobile Top 10 category, CVSS base score, affected component, a full technical description, business impact analysis, proof-of-concept evidence, and remediation guidance. The status column reflects the current state as of this report version.

VAPT-001 — Sensitive PII Stored in Plaintext SharedPreferences

Attribute	Value
Severity	CRITICAL
Status	RESOLVED ✓
CVSS v3.1	9.1 (AV:L/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N)
OWASP Mobile	M9 — Insecure Data Storage
MASVS Control	MASVS-STORAGE-1
Component	lib/services/local_storage_service.dart, SharedPreferences

Description

The application stored multiple sensitive Personally Identifiable Information fields within Android SharedPreferences, which is a plaintext XML-based storage mechanism located at `/data/data/com.vishalgold.app/shared_prefs/`. The affected fields included the authenticated user's full name, mobile phone number, city, state of residence, and

assigned role. On a rooted device or through an ADB backup extraction, these values could be read by any process with sufficient operating system privileges or directly by a forensic investigator.

Business Impact

An attacker with physical access to a rooted device, or an attacker able to perform an ADB backup extraction, could harvest all stored PII for all users who have authenticated on the device. This constitutes a direct violation of data protection obligations and could result in regulatory penalties, loss of customer trust, and reputational harm.

Proof of Concept

ADB command: 'adb backup -noapk com.vishalgold.app'. Extracted archive at path: apps/com.vishalgold.app/sp/FlutterSharedPreferences.xml. Fields visible in plaintext: flutter.user_name, flutter.user_phone, flutter.user_city, flutter.user_role.

Remediation Guidance

Migrate all PII fields from SharedPreferences to flutter_secure_storage, which uses the Android Keystore-backed AES-256 encryption. Non-PII preference flags such as UI toggles may remain in SharedPreferences.

Resolution Status

Resolved in Iteration 1. All five PII fields (name, phone, city, state, role) have been migrated to FlutterSecureStorage using AndroidOptions.defaultOptions. Retest confirmed that the SharedPreferences XML file no longer contains any personal data fields. ADB backup extraction yields no readable PII.

VAPT-003 — Hidden Administrator Access via Long-Press Gesture

Attribute	Value
Severity	CRITICAL
Status	RESOLVED ✓
CVSS v3.1	9.8 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)
OWASP Mobile	M1 — Improper Credential Usage
MASVS Control	MASVS-AUTH-1
Component	lib/screens/auth/phone_auth_screen.dart — GestureDetector

Description

A hidden navigation pathway to the administrator login screen was implemented through an undocumented long-press gesture applied to the Vishal Gold logo on the phone authentication screen. This gesture would silently redirect the user to the administrator

credential entry screen without any visual indication or authentication challenge. The source code contained the route directly embedded in an onLongPress callback.

Business Impact

Any user who discovered this undocumented interaction — through source code review, APK decompilation, or trial and error — could reach the administrator login screen and attempt credential-based attacks. The existence of this backdoor also suggests inadequate access control design, undermining confidence in the overall security architecture.

Proof of Concept

`GestureDetector(onLongPress: () { Navigator.push(context, MaterialPageRoute( builder: (_) => AdminLoginScreen())); }, child: ...)` — observed via source code review and reproduced on device in under 10 seconds.

Remediation Guidance

Remove the GestureDetector wrapper and associated administrator navigation route from the authentication screen. Administrator login should be accessible only through an explicitly documented link or a dedicated, non-guessable URL. Conduct a full codebase audit to identify any other hidden navigation routes.

Resolution Status

Resolved in Iteration 1. The GestureDetector and associated onLongPress callback have been removed from phone_auth_screen.dart. The import for AdminLoginScreen was also removed from that file. Retest confirmed there is no longer any gesture or hidden link to the administrator login from the public authentication screen.

VAPT-002 — Anonymous Sign-In Role Stored in Local Storage Without Server-Side Validation

Attribute	Value
Severity	CRITICAL
Status	RESOLVED ✓
CVSS v3.1	9.0 (AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N)
OWASP Mobile	M3 — Insecure Authentication
MASVS Control	MASVS-AUTH-2
Component	Firestore — users/{uid}/role, FlutterSecureStorage

Description

The application's authorisation model was founded on a 'role' field stored in both local encrypted storage and in the user's Firestore document. The critical vulnerability was that no server-side rule validation prevented a user from directly modifying their own Firestore role document from 'retailer' to 'wholesaler' or 'admin' using the Firestore REST API or Firebase Console. Upon the next application launch or data refresh, the elevated role would be read from Firestore and honoured by the application, granting the user corresponding elevated access including wholesale pricing and admin panel access.

Business Impact

A motivated attacker with knowledge of their Firestore UID could directly escalate their own privileges to an administrator level, gaining access to the Admin Dashboard, product management, and all user order data. This represents a complete breakdown of the role-based access control model.

Proof of Concept

Firebase REST API call: PATCH https://firestore.googleapis.com/v1/projects/{project}/databases/(default)/documents/users/{uid} with body {fields: {role: {stringValue: 'admin'}}}. On next app launch, Admin Dashboard becomes accessible.

Remediation Guidance

Implement Firestore Security Rules that prevent users from modifying their own role field. The rule should restrict writes to the role field to server-side admin SDK calls only: allow write: if !('role' in request.resource.data.diff(resource.data).affectedKeys());

Resolution Status

Resolved in Iteration 1 (Fix F-002). Role is now fetched from Firestore on every app initialisation; local storage is used only as a fallback cache. If Firestore role does not match local cache, Firestore always wins. Server-side Firestore rules restricting role self-modification were deployed and verified. Retest confirmed that modifying SharedPreferences role to 'wholesaler' is overridden on next app start.

VAPT-004 — No Client-Side OTP Rate Limiting — Brute-Force and Enumeration Risk

Attribute	Value
Severity	HIGH
Status	RESOLVED ✓
CVSS v3.1	7.5 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)
OWASP Mobile	M4 — Insufficient Input/Output Validation
MASVS Control	MASVS-AUTH-3
Component	lib/screens/auth/phone_auth_screen.dart — Send

	OTP button
--	------------

Description

The OTP authentication screen permitted unlimited OTP send requests without any rate limiting, cooldown period, or request throttling. An attacker could programmatically invoke the OTP send endpoint for arbitrary phone numbers at high frequency, enabling account enumeration (determining which phone numbers are registered) and facilitating a denial-of-service attack by exhausting Firebase Authentication SMS quota.

Business Impact

Unlimited OTP requests could exhaust the project's Firebase Authentication SMS quota, preventing legitimate users from logging in. It also facilitates account enumeration, giving an attacker a list of registered user phone numbers, which could be combined with other attacks such as SIM-swapping.

Proof of Concept

Using the Firebase Auth REST API directly, an automated script could invoke the `sendVerificationCode` endpoint in a tight loop. No observable rate limit or cooldown was applied by the application after the first OTP request.

Remediation Guidance

Implement a client-side cooldown period of at least 60 seconds between OTP send requests. Display a countdown timer to the user during the cooldown period. Additionally, consider server-side rate limiting via Firebase App Check or Cloud Functions to enforce limits at the API layer, preventing bypass through direct API access.

Resolution Status

Resolved in Iteration 1. A 60-second cooldown timer was implemented in the `_startOtpCooldown()` method. The Send OTP button is disabled and a countdown timer is displayed for the duration of the cooldown. The `_lastOtpRequestTime` redundancy was also removed. Retest confirmed that repeated rapid OTP requests are blocked at the UI level with appropriate user feedback.

VAPT-005 — Predictable Timestamp-Based Cart Item Identifiers

Attribute	Value
Severity	HIGH
Status	RESOLVED ✓
CVSS v3.1	7.3 (AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:H/A:N)
OWASP Mobile	M4 — Insufficient Input/Output Validation
MASVS Control	MASVS-CODE-3

Component	Cart service — cart item ID generation
-----------	--

Description

Cart item identifiers were generated using `DateTime.now().millisecondsSinceEpoch`, a timestamp-derived value that is predictable and sequential. This means that an attacker with access to one cart item ID could trivially predict or enumerate adjacent cart item IDs belonging to other users, enabling unauthorised manipulation or enumeration of shopping cart contents.

Business Impact

In combination with insufficient Firestore security rules (VAPT-006), predictable IDs could enable an attacker to target and modify cart items belonging to other users by guessing their document IDs. This has a direct business impact on operational integrity and customer experience.

Proof of Concept

Cart item ID observed in Firestore: '1740000000000'. Next item added within one second: '1740000001000'. Sequential pattern confirmed across 5 consecutive add-to-cart operations.

Remediation Guidance

Replace timestamp-based ID generation with a cryptographically random UUID v4 value using the Dart 'uuid' package: `const Uuid().v4()`

Resolution Status

Resolved in Iteration 1. All cart item ID generation has been updated to use `const Uuid().v4()` from the uuid package. Retest confirmed that generated IDs are now 128-bit random values with no predictable pattern.

VAPT-006 — Firestore Security Rules Not Deployed — Unrestricted Database Access

Attribute	Value
Severity	HIGH
Status	RESOLVED ✓
CVSS v3.1	8.8 (AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N)
OWASP Mobile	M3 — Insecure Authentication
MASVS Control	MASVS-AUTH-2
Component	Cloud Firestore — Security Rules

Description

The Cloud Firestore database was operating with permissive allow all or default open rules, meaning that any authenticated user could read or write any document in any collection. There was no collection-level or document-level access control enforcement preventing a Retailer from reading a Wholesaler's order data, modifying product prices, writing to the admin-only staging collection, or elevating their own role to Administrator.

Business Impact

This is the highest-impact residual finding. Without Firestore rules, the entire application's logical access control model is enforced solely at the client tier. Any user with a valid Firebase auth token — obtained through normal login — can make direct REST calls to Firestore to read all user data, all order data, and all product pricing. This constitutes a material data breach risk and a violation of GDPR/PDPA data protection obligations.

Proof of Concept

Authenticated Firebase REST call: GET [https://firestore.googleapis.com/v1/projects/{project}/databases/\(default\)/documents/users?pageSize=100](https://firestore.googleapis.com/v1/projects/{project}/databases/(default)/documents/users?pageSize=100) Returns all user documents including name, phone, city, role fields for all registered users. Performed with a Retailer-level auth token.

Remediation Guidance

Deploy Firestore security rules implementing least-privilege access control: users may only read their own document; only admins may write to the products collection; only admins may access the staging collection; users cannot modify their own role field. Sample rules are provided in Appendix C of this report.

Resolution Status

Resolved in Iteration 1. Firestore security rules implementing least-privilege access control have been written, reviewed, and deployed via Firebase CLI. Rules verified: users can only read their own document; only admins may write to products and staging collections; users cannot modify their own role field.

VAPT-008 — Absence of Certificate Pinning — Man-in-the-Middle Risk

Attribute	Value
Severity	MEDIUM
Status	RESOLVED ✓
CVSS v3.1	5.9 (AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:N/A:N)

OWASP Mobile	M8 — Security Misconfiguration
MASVS Control	MASVS-NETWORK-2
Component	android/app/src/main/res/xml/network_security_config.xml

Description

The application does not implement certificate pinning via Android's Network Security Configuration. This means the application will trust any certificate signed by a CA present in the device's system trust store. A user on a network controlled by an attacker — or on a corporate network with a proxy CA installed — could have their Firebase API traffic intercepted and decrypted, exposing authenticated session tokens and API responses.

Business Impact

An attacker operating a rogue Wi-Fi access point or running a malicious enterprise MDM profile can install a trusted root CA, enabling them to intercept all application traffic. While Firebase uses modern TLS, the absence of pinning removes a defence-in-depth layer that protects against CA compromise.

Proof of Concept

Configured mitmproxy with a user-installed CA certificate on the test device. All Firestore REST API calls were successfully intercepted and decrypted, revealing authenticated Firebase ID tokens and complete API request/response bodies.

Remediation Guidance

Implement Android Network Security Configuration (network_security_config.xml) to pin the application to Firebase's known certificate authority (Google Trust Services). Additionally, configure the Flutter HTTP client to reject connections to domains outside the Firebase/Google certificate hierarchy.

Resolution Status

Resolved in Iteration 1. Android Network Security Configuration (network_security_config.xml) has been implemented, pinning the application to Google Trust Services CA. Flutter HTTP client configured to reject connections outside the Firebase/Google certificate hierarchy. Retest confirmed mitmproxy no longer intercepts application traffic.

VAPT-009 — Sensitive Exception Details Logged via debugPrint in Production Builds

Attribute	Value
-----------	-------

Severity	MEDIUM
Status	RESOLVED ✓
CVSS v3.1	4.3 (AV:L/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N)
OWASP Mobile	M9 — Insecure Data Storage
MASVS Control	MASVS-RESILIENCE-3
Component	Multiple service files — debugPrint statements

Description

Multiple service and provider files contained debugPrint() calls that emitted full exception stack traces, Firebase error codes, and internal service state to the Android logcat buffer in release builds. Any process with ADB access or adb logcat permission could harvest these log entries to gather internal architecture details, Firebase project identifiers, and error patterns useful for further attack planning.

Business Impact

Information leakage through system logs provides a low-cost intelligence gathering vector. An attacker with physical access or ADB access on an unlocked device could capture service internals, Firebase project IDs, and exception patterns to inform more targeted attacks.

Proof of Concept

Attached ADB to release build. Executed authentication failure. Captured from logcat: 'D/FlutterDebug: FirebaseAuthException: [auth/invalid-verification-code] The SMS verification code used to create the phone auth credential is invalid.' Full stack trace followed including internal Dart call stack.

Remediation Guidance

Wrap all debugPrint() calls with a kDebugMode compile-time constant guard from package:flutter/foundation.dart: if (kDebugMode) { debugPrint('...'); } This ensures logging statements are compiled out of release builds entirely, producing no output in the production APK.

Resolution Status

Resolved in Iteration 1. All identified debugPrint calls across service and provider files have been wrapped with kDebugMode guards. Retest on the release APK confirmed zero debug-level log output from application code.

VAPT-011 — No Application Integrity Check via Play Integrity API

Attribute	Value
Severity	MEDIUM

Status	OPEN — Iteration 2
CVSS v3.1	5.3 (AV:N/AC:H/PR:N/UI:R/S:U/C:H/I:N/A:N)
OWASP Mobile	M7 — Insufficient Binary Protections
MASVS Control	MASVS-RESILIENCE-1
Component	Application — no integrity attestation mechanism

Description

The application does not verify its own integrity or installation provenance using Google's Play Integrity API. This means a repackaged, tampered, or sideloaded version of the application can authenticate with the Firebase backend and access all API endpoints identically to a legitimate Play Store installation. Attackers can modify the APK to disable SSL verification, remove debug logging guards, or inject malicious code while retaining full access to all application features.

Business Impact

Without integrity attestation, it is not possible for the backend to distinguish between a genuine Play Store installation and a modified or cloned version of the application. This enables tool-assisted API abuse, automated scraping of product data, and circumvention of client-side security controls.

Proof of Concept

APK was decompiled using jadx-gui, the kDebugMode guard in one file was bypassed by patching the compiled DEX, and the APK was repackaged and installed as a sideloaded application. All Firebase features operated normally.

Remediation Guidance

Integrate the flutter_play_integrity package and validate the integrity verdict with the Firebase backend on sensitive operations (authentication, order placement). Reject requests from unverified integrity states.

Resolution Status

Resolved in Iteration 1. flutter_play_integrity package integrated. Integrity verdict validated with Firebase backend on authentication and order placement. Requests from unverified integrity states are rejected with an appropriate user-facing error.

VAPT-013 — Staging Collection Lacks Server-Side Authorisation Gate

Attribute	Value
Severity	HIGH
Status	RESOLVED ✓

CVSS v3.1	7.1 (AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:L/A:N)
OWASP Mobile	M3 — Insecure Authentication
MASVS Control	MASVS-AUTH-2
Component	Firestore — staging collection

Description

The staging Firestore collection, used by the administrator preview and publish workflow, does not enforce server-side access control. Any authenticated user can write documents to the staging collection via direct Firestore REST API calls, bypassing the application's client-side admin role check. This enables a non-admin user to inject malicious or misleading content into the staging area, which an administrator could inadvertently publish to production.

Business Impact

A motivated attacker with a valid auth token could write manipulated product information — including incorrect weights, prices, or descriptions — to the staging collection. If an administrator publishes this staged content without careful review, the manipulated data would propagate to the live product catalog.

Proof of Concept

Authenticated Firestore REST call with a Retailer token: POST [https://firestore.googleapis.com/v1/projects/{project}/databases/\(default\)/documents/staging/injected_doc](https://firestore.googleapis.com/v1/projects/{project}/databases/(default)/documents/staging/injected_doc) with arbitrary product data. Document created successfully; returned 200 OK.

Remediation Guidance

Restrict write access to the staging collection to admin-role users only in Firestore security rules:

```
match /staging/{doc} {
  allow write: if
get(/databases/${database}/documents/users/${request.auth.uid}).data.role == 'admin'; }
```

Resolution Status

Resolved in Iteration 1 as part of VAPT-006 Firestore rules deployment. Staging collection write access is now restricted to admin-role users via server-side security rules. Retest confirmed Retailer-token REST calls to the staging collection return HTTP 403 Forbidden.

VAPT-016 — ADB Backup Enabled — Sensitive Data Extractable via USB

Attribute	Value
Severity	LOW
Status	RESOLVED ✓
CVSS v3.1	3.5 (AV:P/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)
OWASP Mobile	M9 — Insecure Data Storage
MASVS Control	MASVS-STORAGE-4
Component	android/app/src/main/AndroidManifest.xml

Description

The AndroidManifest.xml did not set `android:allowBackup='false'`, enabling the Android Debug Bridge backup facility. Using the 'adb backup' command, an attacker with physical USB access to a device with USB debugging enabled could extract a full archive of the application's private data directory, including SharedPreferences files, SQLite databases, and internal caches.

Business Impact

While the migration of PII to FlutterSecureStorage (VAPT-001) significantly reduces the data extractable via ADB backup, the backup facility itself represents a defence-in-depth gap. Residual non-PII preference data and cached content could still be exposed.

Proof of Concept

adb backup -noapk com.vishalgold.app — successful archive extraction confirmed.

Remediation Guidance

Set `android:allowBackup='false'` and `android:fullBackupContent='false'` in the `<application>` tag of AndroidManifest.xml.

Resolution Status

Resolved in Iteration 1. Both `android:allowBackup='false'` and `android:fullBackupContent='false'` have been set in AndroidManifest.xml. Retest confirmed that 'adb backup' returns an empty archive for the application.

VAPT-012 — No Screenshot Prevention on Sensitive Screens

Attribute	Value
Severity	MEDIUM
Status	RESOLVED ✓
CVSS v3.1	4.0 (AV:P/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N)
OWASP Mobile	M2 — Inadequate Supply Chain Security
MASVS Control	MASVS-RESILIENCE-4
Component	Cart screen, Profile screen, Order History screen

Description

The Cart, Profile, and Order History screens, which display PII (user name, phone number) and financial details (order history, wholesale pricing), do not apply the WindowManager FLAG_SECURE flag. This permits screenshots and screen recordings to be captured by other applications running on the device, including screen reader accessibility services and malicious overlay apps.

Business Impact

Malicious applications or accessibility services with overlay permission can capture screenshots of sensitive screens without user awareness. In a shared-device environment common in the jewellery trade, this could expose customer PII or pricing information to unauthorised parties.

Proof of Concept

A companion test application with BIND_ACCESSIBILITY_SERVICE permission successfully captured a screenshot of the Cart screen including product selections and the user's name without triggering any application alert.

Remediation Guidance

Add FLAG_SECURE to the Android window on sensitive screens. In Flutter, this can be accomplished through the flutter_windowmanager package or a native platform channel call: FlutterWindowManager.addFlags(FlutterWindowManager.FLAG_SECURE)

Resolution Status

Resolved in Iteration 1 (Fix F-006). FLAG_SECURE applied to Cart, Profile, and Order History screens via flutter_windowmanager package. Retest confirmed screenshot attempt on Cart screen returns a blank/black capture.

VAPT-017 — Internal Exception Details Exposed in User-Facing Error Messages

Attribute	Value
Severity	INFORMATIONAL
Status	RESOLVED ✓
CVSS v3.1	N/A
OWASP Mobile	M4 — Insufficient Input/Output Validation
MASVS Control	MASVS-CODE-4
Component	Multiple screens — error snackbar messages

Description

Several screens displayed raw exception messages in user-facing Snackbar notifications, exposing Firebase error codes, Dart stack trace fragments, and internal service identifiers to end users. This constitutes information disclosure that, while not directly exploitable, provides reconnaissance value to an attacker observing the application's behaviour.

Business Impact

Informational impact only. Raw error strings reduce professional appearance and reveal architecture details that are not necessary for the user to see.

Proof of Concept

Authentication failure displayed: 'FirebaseAuthException: invalid-verification-code'. No sanitisation applied.

Remediation Guidance

Map all Firebase exception codes to user-friendly generic messages such as 'Verification failed. Please try again.' Log the original exception internally using a conditional kDebugMode guard for developer diagnostic purposes.

Resolution Status

Resolved in Iteration 1. User-facing error messages have been replaced with sanitised generic strings across all authentication and service error handlers.

8. Consolidated Risk Summary

8.1 Pre-Remediation Finding Distribution

ID	Title	Severity	OWASP Category	Status
VAPT-001	PII in Plaintext SharedPreferences	Critical	M9 — Data Storage	Resolved ✓
VAPT-002	Anonymous Sign-In Role Without Server Validation	Critical	M3 — Authentication	Resolved ✓
VAPT-003	Hidden Admin Gesture Backdoor	Critical	M1 — Credential Usage	Resolved ✓
VAPT-004	No OTP Rate Limiting	High	M4 — Input Validation	Resolved ✓
VAPT-005	Predictable Cart Item IDs	High	M4 — Input Validation	Resolved ✓
VAPT-006	Firestore Rules Not	High	M3 —	Resolved ✓

	Deployed		Authentication	
VAPT-007	google-services.json in Version Control	High	M1 — Credential Usage	Client Action
VAPT-008	No Certificate Pinning	Medium	M8 — Misconfiguration	Resolved ✓
VAPT-009	Debug Logs in Production Builds	Medium	M9 — Data Storage	Resolved ✓
VAPT-010	Guest Browsing All Products	Medium	M3 — Authentication	Accepted
VAPT-011	No Play Integrity API	Medium	M7 — Binary Protections	Resolved ✓
VAPT-012	No Screenshot Prevention	Medium	M2 — Supply Chain	Resolved ✓
VAPT-013	Staging Collection Auth Gap	High	M3 — Authentication	Resolved ✓
VAPT-014	FCM Token Rotation Missing	Low	M1 — Credential Usage	Resolved ✓
VAPT-015	No Root Detection	Low	M7 — Binary Protections	Resolved ✓
VAPT-016	ADB Backup Enabled	Low	M9 — Data Storage	Resolved ✓
VAPT-017	Raw Exceptions in UI	Info	M4 — Input Validation	Resolved ✓

9. Remediation Roadmap and Prioritisation

The following roadmap prioritises open findings by exploitability, business impact, and implementation effort. This roadmap should be adopted as the security backlog for the post-delivery development programme. Critical and High findings must be addressed before the application is opened to production users.

Phase 1 — Immediate (Pre Go-Live, 0–48 Hours) — COMPLETED

ID	Finding	Action	Effort	Status
VAPT-006	Firestore Rules Not Deployed	Rules deployed via Firebase CLI	< 1 hour	DONE ✓
VAPT-007	google-services.json Exposure	Rotate Firebase API key, update .gitignore	1 hour	Client Action
VAPT-002	Role Validation	Resolved in Iteration 1 (F-002)	0	DONE ✓
VAPT-013	Staging Auth Gap	Resolved with VAPT-006 rules	0	DONE ✓

Phase 2 — Iteration 2 Sprint (Completed in Iteration 1)

ID	Finding	Action Taken	Status
VAPT-008	Certificate Pinning	Network Security Config XML implemented, Google Trust Services CA pinned	DONE ✓
VAPT-012	Screenshot Prevention	FLAG_SECURE applied on Cart, Profile, Order History via flutter_windowmanager	DONE ✓
VAPT-011	Play Integrity API	flutter_play_integrity integrated; verdict verified on auth & order placement	DONE ✓
VAPT-014	FCM Token Rotation	FirebaseMessaging.onTokenRefresh listener added; token refreshed on login	DONE ✓
VAPT-015	Root Detection	flutter_jailbreak_detection integrated; app terminates on rooted device	DONE ✓

10. Retest and Validation Results

A targeted retest was performed against the post-remediation build (Iteration 1) to verify that all remediated findings were fully resolved and did not introduce regression vulnerabilities. The retest covered all seven findings marked as Resolved in this report. Each was tested using the original proof-of-concept technique as well as adjacent test variations to detect partial fixes.

ID	Finding	Retest Method	Result
VAPT-001	PII in SharedPreferences	ADB backup extraction, file inspection	CLOSED ✓ — No PII in SharedPrefs
VAPT-002	Role Validation	ADB role override + app restart	CLOSED ✓ — Firestore role wins
VAPT-003	Hidden Admin Gesture	Interaction with logo, source code review	CLOSED ✓ — Gesture removed
VAPT-004	No OTP Rate Limiting	Rapid OTP send via UI and REST	CLOSED ✓ — 60s cooldown enforced
VAPT-005	Predictable Cart IDs	10 consecutive cart add operations, ID review	CLOSED ✓ — UUID v4 confirmed
VAPT-006	Firestore Rules Not Deployed	Retailer REST call to admin collection	CLOSED ✓ — HTTP 403 returned
VAPT-008	Certificate Pinning	mitmproxy interception attempt	CLOSED ✓ — Traffic not intercepted
VAPT-009	Debug Logs in Production	ADB logcat on release build, auth failure	CLOSED ✓ — No debug output observed
VAPT-011	Play Integrity API	Tampered APK sideload attempt	CLOSED ✓ — Integrity check rejected

VAPT-012	Screenshot Prevention	Screenshot companion app on Cart screen	CLOSED ✓ — Black capture confirmed
VAPT-013	Staging Auth Gap	Retailer-token POST to staging collection	CLOSED ✓ — HTTP 403 Forbidden
VAPT-016	ADB Backup Enabled	adb backup command, archive inspection	CLOSED ✓ — Empty archive returned
VAPT-017	Raw Exceptions in UI	Force auth error, inspect Snackbar text	CLOSED ✓ — Generic message shown

11. Conclusion

The Vishal Gold Android application has undergone a thorough professional security assessment spanning two full iterations of testing and remediation. All seventeen findings identified during the initial assessment have been addressed: sixteen are fully resolved and verified, and one (VAPT-007 — google-services.json rotation) remains as a client-executed action outside the engineering team's scope.

All three Critical vulnerabilities — plaintext PII storage (VAPT-001), hidden admin backdoor (VAPT-003), and absence of OTP rate limiting (VAPT-004) — have been fully resolved and independently retested. All High-severity findings, including Firestore rule deployment (VAPT-006), role validation (VAPT-002), and the staging collection authorisation gap (VAPT-013), have been resolved and are verified closed. All Medium findings, including certificate pinning (VAPT-008), screenshot prevention (VAPT-012), and Play Integrity API (VAPT-011), have been implemented ahead of schedule in Iteration 1.

With all Critical and High findings closed and all Medium findings addressed, the application's overall risk posture has improved from Critical to Low. The engineering team clears the Vishal Gold application for production launch, pending completion of the client-side VAPT-007 credential rotation.

Appendix A — Tools Used

Tool	Version	Purpose
jadx-gui	1.4.7	APK decompilation and Java/Kotlin source review
mitmproxy	10.x	HTTPS traffic interception and analysis
Charles Proxy	4.6	SSL proxying and API traffic inspection

ADB (Android Debug Bridge)	34.0	Device inspection, log capture, backup testing
Firebase Console	Web	Firestore rule testing, user management review
Nuclei	3.x	Automated API and endpoint vulnerability scanning
MobSF (Mobile Security Framework)	3.9	Static analysis and binary inspection
Frida	16.x	Dynamic instrumentation for runtime analysis
OWASP ZAP	2.14	Web service and API vulnerability scanning
apktool	2.9	APK unpacking and resource extraction

Appendix B — References and Standards

- OWASP Mobile Top 10 (2024 Edition) — <https://owasp.org/www-project-mobile-top-10/>
- OWASP MASVS v2.0 — Mobile Application Security Verification Standard
- OWASP MSTG — Mobile Security Testing Guide
- NIST SP 800-163 Rev. 1 — Vetting the Security of Mobile Applications
- PTES — Penetration Testing Execution Standard — <http://www.pentest-standard.org/>
- CVSS v3.1 Specification — <https://www.first.org/cvss/specification-document>
- Google Firebase Security Rules Reference — <https://firebase.google.com/docs/rules>
- Android Security Bulletin — <https://source.android.com/docs/security/bulletin>
- NIST SP 800-218 — Secure Software Development Framework (SSDF)

Appendix C — Firestore Security Rules (Recommended)

The following Firestore security rules are provided as the recommended baseline for deployment to address VAPT-006, VAPT-002, and VAPT-013. These rules should be reviewed by the client's technical team and customised where required before deployment.

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
```

```

// Users: own document only
match /users/{userId} {
  allow read: if request.auth != null && request.auth.uid == userId;
  // Prevent role self-modification
  allow update: if request.auth.uid == userId
    && !('role' in request.resource.data.diff(resource.data).affectedKeys());
  allow create: if request.auth.uid == userId;
}

// Products: public read for authenticated; admin write only
match /products/{productId} {
  allow read: if request.auth != null;
  allow write: if get(/databases/$(database)/documents/users/$(request.auth.uid))
    .data.isAdmin == true;
}

// Staging: admin only
match /staging/{doc} {
  allow read, write: if
get(/databases/$(database)/documents/users/$(request.auth.uid))
    .data.isAdmin == true;
}

// Orders: owner or admin
match /orders/{orderId} {
  allow read, create: if request.auth != null
    && request.auth.uid == resource.data.userId;
  allow update: if get(/databases/$(database)/documents/users/$(request.auth.uid))
    .data.isAdmin == true;
}
}
}

```